

# Network Security

**Theory Class Nº7**

Jorge Sa Silva
sasilva@deec.uc.pt
University of Coimbra
Portugal

# Outline

- Cryptography
  - Transport Layer Security (TLS)
    - Message Authentication Code (MAC)
    - RSA
    - Trusted certificates
  - Routing (revision)
  - Darkweb/Deepweeb

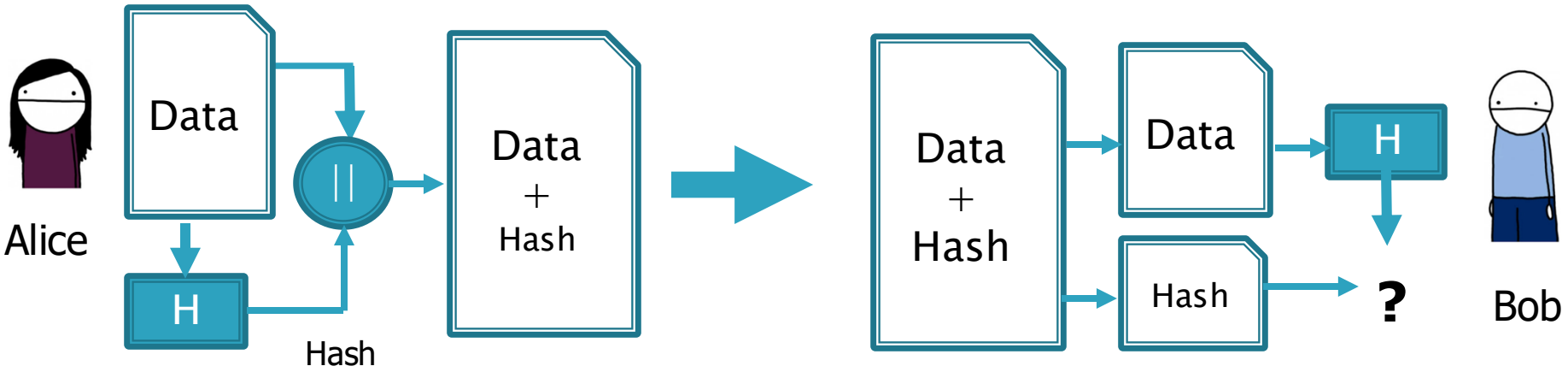


# Cryptography

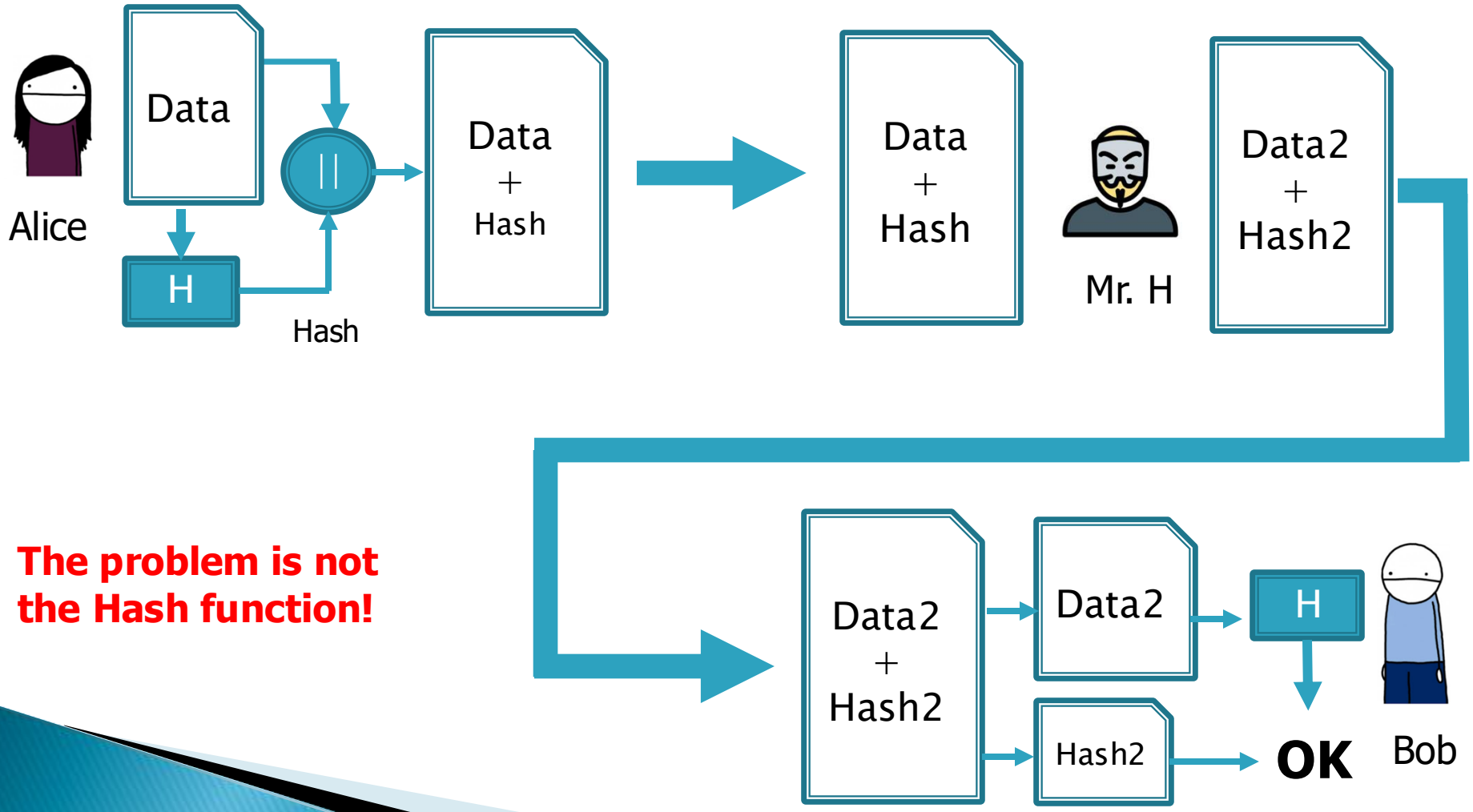
- **Perspective**
  - User
  - Manager
  - Developer
- **Ingredients**
  - Requirements:
    - Data integrity
    - User authentication
    - Confidentiality
  - Kerckhoff's principle
  - Random and pseudo-random generators
  - Symmetric Encryption
  - Asymmetric Encryption
  - Hash
  - Diffie-Hellman
  - Message Authentication Code (MAC)
  - RSA
  - Digital certificates
  - …

- **Applications**
  - Digital signatures
  - TLS
  - Darkweb
  - Blockchain
  - Firewall
  - VPNs
  - …

# Applications of Hash functions – Integrity



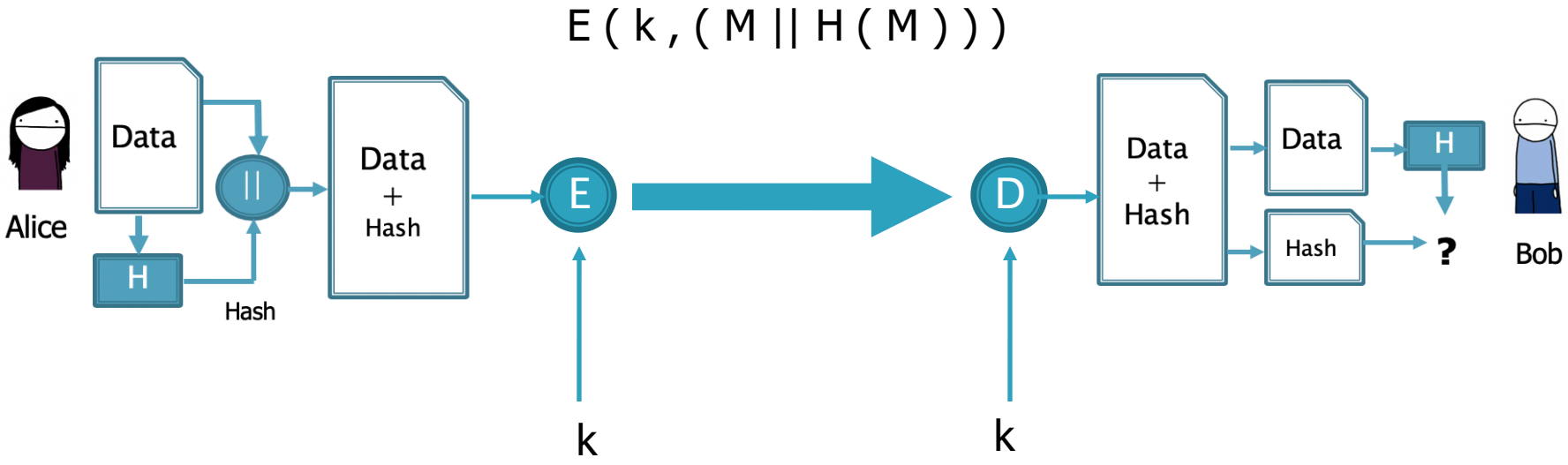
# Man-in-the-middle attack




**The problem is not the Hash function!**

# Applications of Hash functions

## Integrity + Authentication + Confidentiality

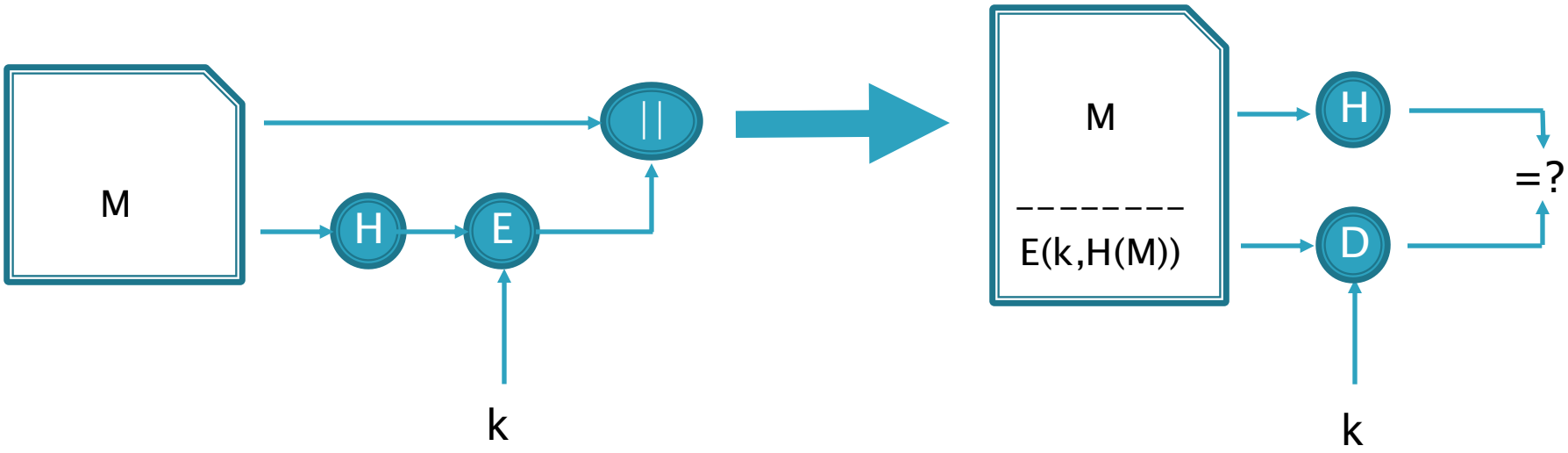



K – shared key

# Applications of Hash functions

## Integrity + Authentication

- Confidentiality not necessary

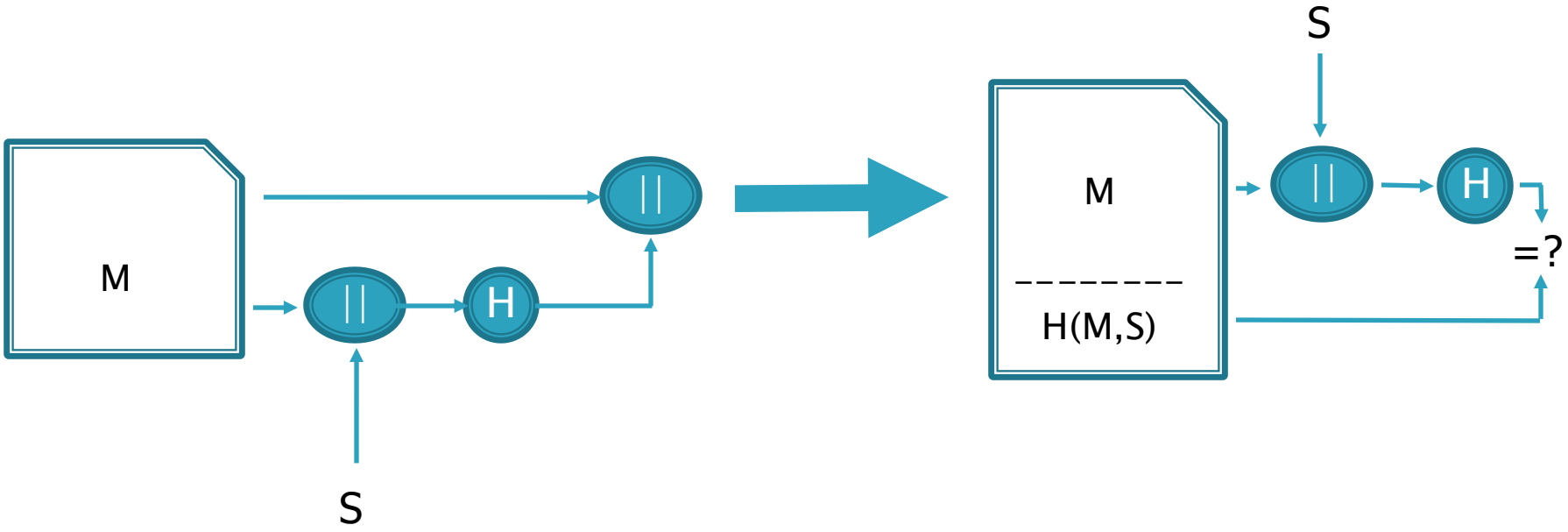



K – shared key

# Applications of Hash functions

## Integrity + Authentication

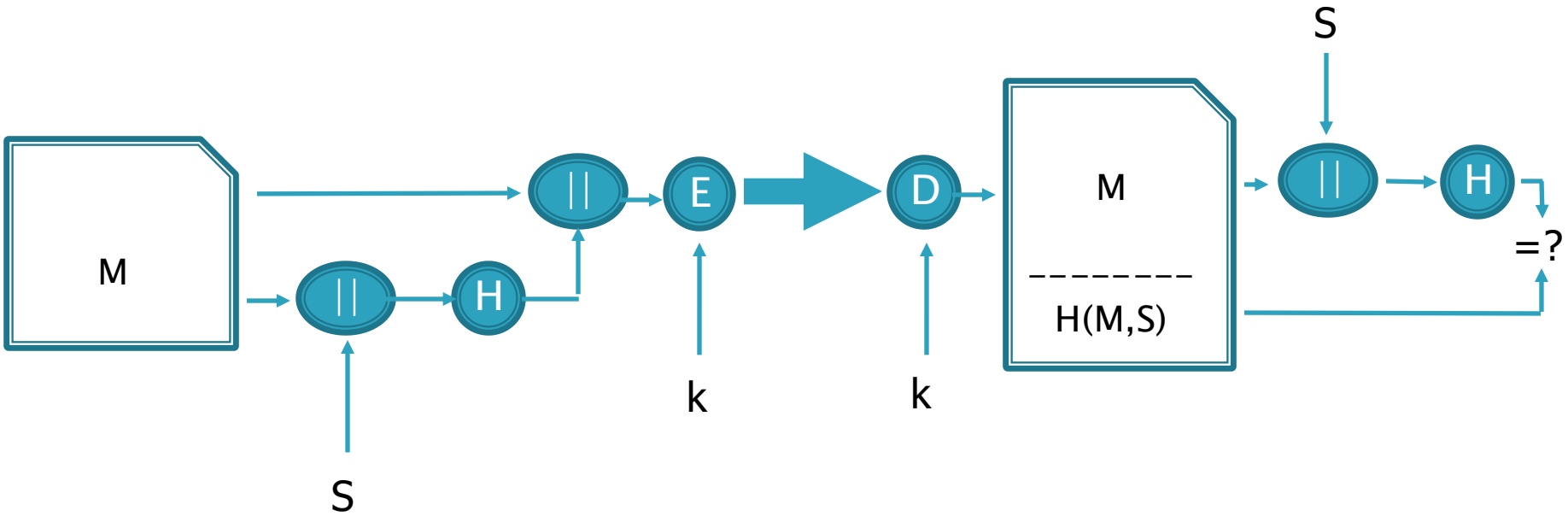
- Confidentiality not necessary
- Use of secret code (S) instead of encryption/decryption



# Applications of Hash functions

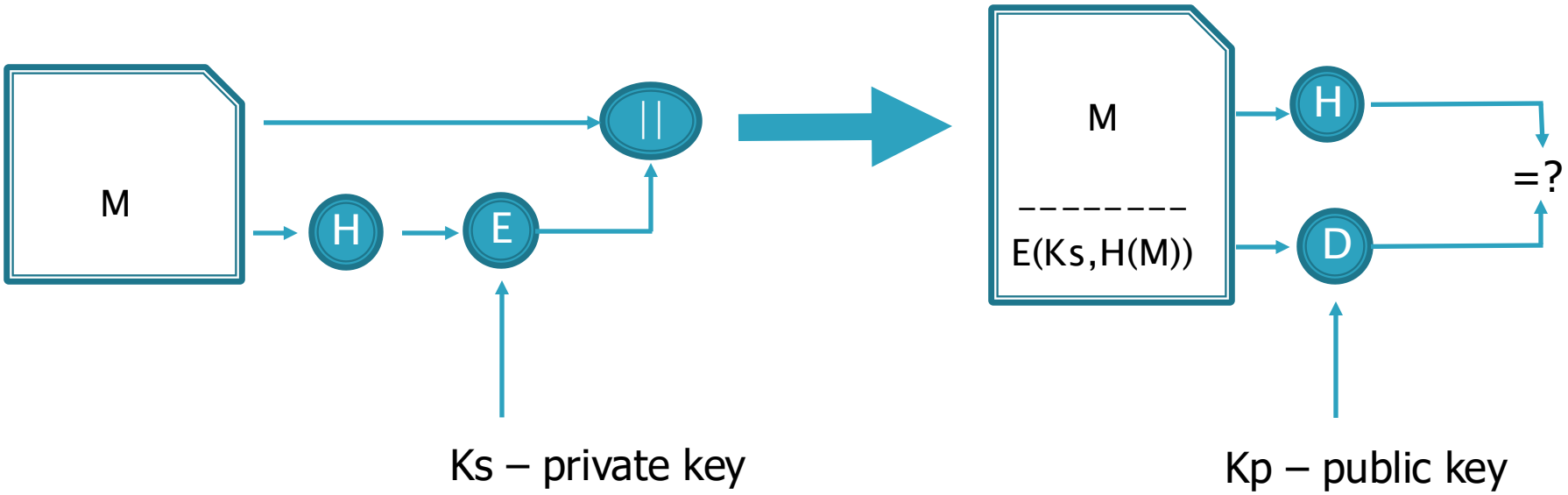
## Integrity + Authentication + Confidentiality

- Use of secret code (S) and encryption(E) / decryption(D)



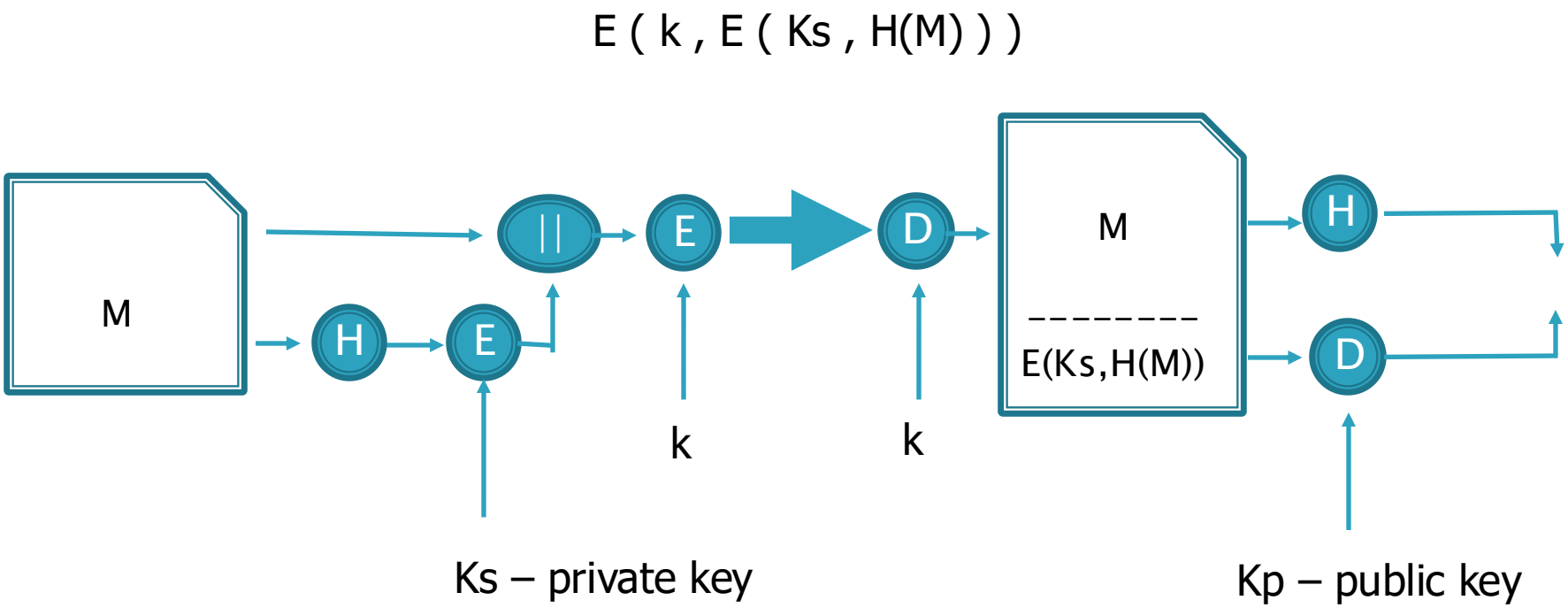

K – shared key

# Applications of Hash functions
## Digital signatures
### Integrity + Authentication



# Applications of Hash functions
## Digital signatures

### Integrity + Authentication + Confidentiality

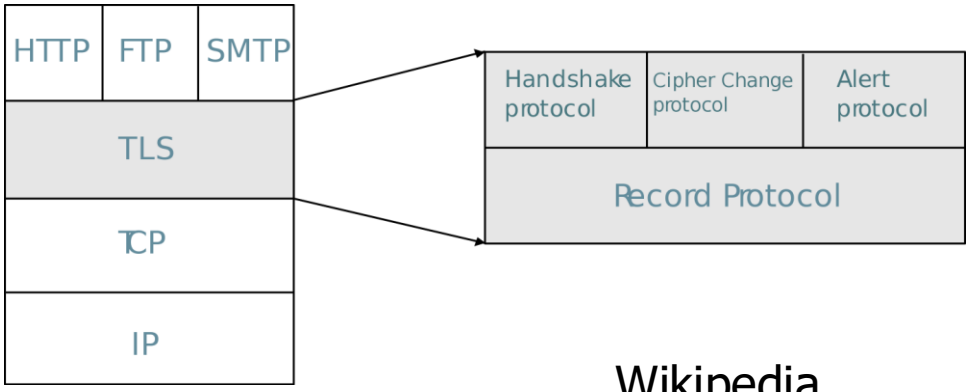


# Hash functions – other applications

- Files with passwords
  - Usually OS do not save password but use their hash codes
- Intrusion detection in files
- Virus detection
- Generation of pseudorandom numbers

# Transport Layer Security (TLS)

- TLS is the successor of the now-deprecated Secure Sockets Layer (SSL)
- TLS is a cryptographic protocol designed to provide communications security over a computer network.
- The protocol is widely used in applications such as email, instant messaging, and voice over IP, but its use in securing HTTPS remains the most publicly visible.




Wikipedia

# History of TLS

- SSL 1.0
  - Internal Netscape design, early 1994?
  - Lost in the mists of time
- SSL 2.0
  - Published by Netscape, November 1994
  - Badly broken
- SSL 3.0
  - Designed by Netscape and Paul Kocher, November 1996
- TLS 1.X
  - Internet standard based on SSL 3.0, January 1999
  - Not interoperable with SSL 3.0
  - Current version: TLS 1.3 defined in RFC 8446

# Transport Layer Security (TLS)

- **Confidentiality**: Encrypts data to prevent unauthorized access.
- **Integrity**: Ensures data isn't altered during transit.
- **Authentication**: Verifies the identity of the server (and optionally the client) using certificates.



**1) Handshake Process**:

- **Initiation**: When a client (e.g., your browser) connects to a server (e.g., a website), it starts the TLS handshake. The client sends a "ClientHello" message, specifying supported TLS versions and cryptographic algorithms (cipher suites).
- **Server Response**: The server responds with a "ServerHello" message, selecting a compatible TLS version and cipher suite. It also sends its **digital certificate**, which contains its **public key** and is issued by a trusted Certificate Authority (CA).
- **Authentication**: The client verifies the server's certificate by checking it against trusted CAs to ensure the server is legitimate.
- **Key Exchange**: The client and server use a key exchange algorithm (e.g., Diffie-Hellman or RSA) to securely agree on a **symmetric session key**. This key will be used for encrypting data during the session. The process ensures that even if someone intercepts the handshake, they can't derive the session key.
- **Completion**: Both sides confirm the handshake is complete, and the secure connection is established.

**2) Data Encryption**:

- Once the handshake is done, all data exchanged is encrypted using the symmetric session key (typically using algorithms like AES). Symmetric encryption is fast and efficient for bulk data transfer.
- TLS also ensures **data integrity** using message authentication codes (MACs), which detect any tampering with the data.

**3) Session Communication**:

- The client and server now communicate securely, with all data encrypted and protected from eavesdropping or alteration. For example, when you visit an HTTPS website, TLS ensures your passwords, credit card details, or other sensitive data remain private.

**4) Termination**:

- When the session ends (e.g., you close the browser tab), the connection is terminated, and the session key is discarded. A new handshake is required for future connections.

# Transport Layer Security (TLS)

- Since applications can communicate either with or without TLS (or SSL), it is necessary for the client to request that the server sets up a TLS connection.[
- One of the main ways of achieving this is to use a different port number for TLS connections. For example, port 80 is typically used for unencrypted HTTP traffic while port 443 is the common port used for encrypted HTTPS traffic.
- Another mechanism is for the client to make a protocol-specific request to the server to switch the connection to TLS; for example, by making a STARTTLS request when using the mail and news protocols.

# TLS Basics

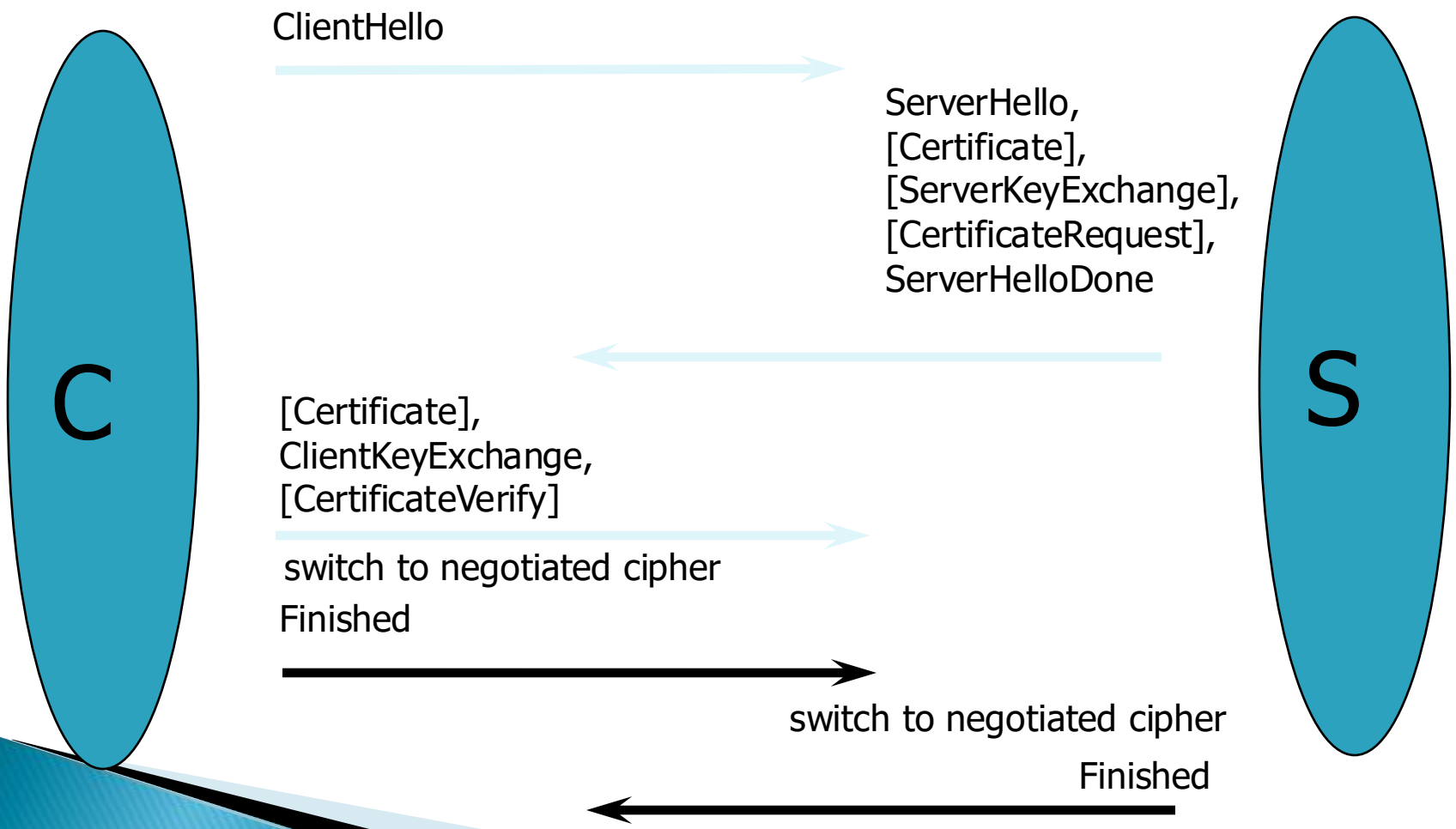
- TLS consists of two protocols
  - Handshake protocol
    - Use public-key cryptography to establish a shared secret key between the client and the server
  - Record protocol
    - Use the secret key established in the handshake protocol to protect communication between the client and the server

# TLS Handshake Protocol



- Two parties: client and server
- Negotiate version of the protocol and the set of cryptographic algorithms to be used
  - Interoperability between different implementations of the protocol
- Authenticate client and server (optional)
  - Use digital certificates to learn each other's public keys and verify each other's identity
- Use public keys to establish a shared secret

# Handshake Protocol Structure



# Abbreviated Handshake

- The handshake protocol may be executed in an abbreviated form to resume a previously established session
  - No authentication, key material not exchanged
  - Session resumed from an old state

# Trusted certificate

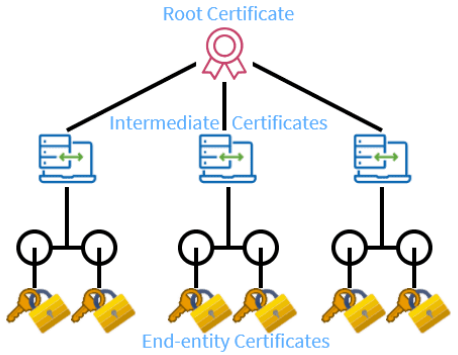
- It contains:
  - The **name of the entity** (e.g., www.uc.pt);
  - The **public key** of the entity;
  - The **name of the issuing authority** (CA — *Certificate Authority*);
  - A **validity period**;
  - And a **digital signature** from the CA.

# How it works

- When a client (for example, your browser) connects to an HTTPS server:
- The server sends its **digital certificate.**
- The client verifies:
  - Whether the certificate **has not expired**;
  - Whether the **domain name matches** (e.g., www.uc.pt);
  - Whether it was **signed by a trusted CA.**
- If the certificate was signed by an **intermediate CA**, the client checks the **certificate chain** until it reaches a **trusted root CA.**
- If everything is valid, the communication is considered **secure**, and the client can establish an **encrypted channel (TLS/SSL).**

# Trusted certificate

- **Certificate Authority (CA)**: A trusted third-party organization that issues digital certificates after verifying a website's identity.
- **Root Certificate**: A self-signed certificate that is the top of the "chain of trust". Every major operating system and browser comes with a built-in list of trusted root certificates from well-known CAs.
- **Chain of Trust**: When you visit a website with an SSL/TLS certificate, your browser checks the certificate's signature to see if it can be traced back through a series of "intermediate certificates" to a root certificate already in its trust store.
- **Validation**: If the chain is valid, your browser trusts the website and establishes a secure, encrypted connection.



# Message Authentication Code (MAC)

- A cryptographic technique that uses a symmetric key to verify both the data integrity and authenticity of a message.
- It generates a short, fixed-size "tag" that can be used to confirm that the message came from the correct sender and has not been altered.
- MACs are essential for secure communication because they protect against tampering and ensure that the data is legitimate.

# How MACs work

- **Secret key**: A secret, symmetric key must be shared between the sender and the intended recipient beforehand.
- **Generation**: The sender combines the message with the secret key using a MAC algorithm to produce a unique authentication tag.
- **Verification**: The recipient uses the same secret key and MAC algorithm to generate a new tag from the received message. If the recipient's calculated tag matches the sender's tag, the message is considered authentic and unaltered.

# Key properties and use cases

- **Authenticity**: MACs verify the origin of a message, as only someone with the secret key can generate a valid MAC tag.
- **Integrity**: They ensure the message has not been modified in transit, as any change to the message would result in a different MAC tag.
- **Symmetric**: Both the sender and receiver use the same secret key to generate and verify the MAC.
- **Speed**: They are faster than digital signatures, making them ideal for high-speed applications and secure communication protocols like TLS and SSH.

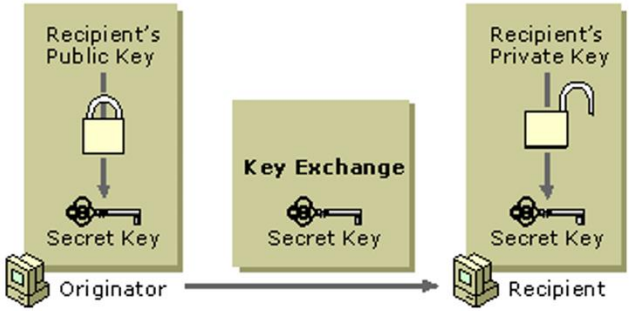
# MACs vs. other cryptography

- **Vs. hash functions**: A simple hash function only verifies data integrity; it does not guarantee authenticity because anyone can compute a hash of a message. MACs add the secret key to provide this crucial authentication layer.
- **Vs. digital signatures**: Digital signatures use asymmetric cryptography to provide authenticity and integrity, but they also offer non-repudiation, meaning the sender cannot deny sending the message. MACs, using symmetric keys, do not provide non-repudiation.

# Rivest–Shamir–Adleman (RSA) algorithm



- **Public and private keys**: It uses a pair of keys: a public key that can be shared and a private key that must be kept secret.
- **Key exchange**: One party generates a secret symmetric key (e.g., AES).
- **Encryption**: This secret key is then encrypted using the other party's public RSA key.
- **Transmission**: The encrypted secret key is sent to the other party over an insecure channel.
- **Decryption**: The recipient uses their unique private key to decrypt the message and retrieve the original secret key.
- **Shared secret**: Both parties now have the same secret key and can use it for efficient symmetric encryption for their session.





# Routing

# TCP/IP



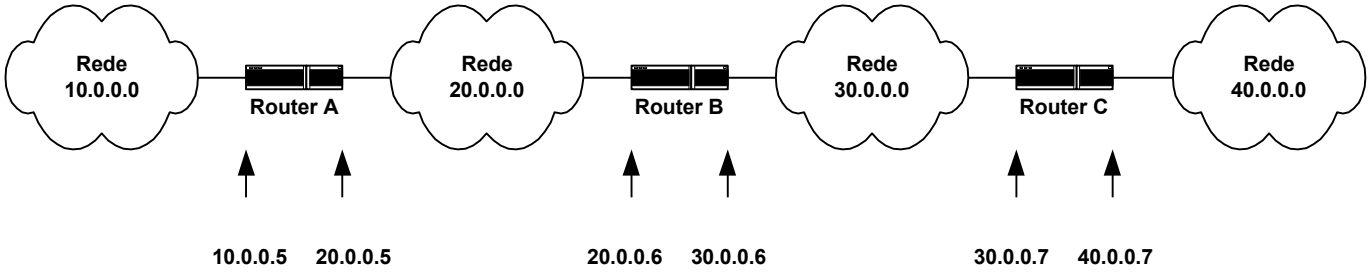
Camada de Aplicação

Camada de Transporte

Camada de Rede

Camada de Acesso ao Meio

# Routing




**Router B**

| Estação na rede | Encaminhamento |
|---|---|
| 20.0.0.0 | Directo |
| 30.0.0.0 | Directo |
| 10.0.0.0 | 20.0.0.5 |
| 40.0.0.0 | 30.0.0.7 |

# Equipamento de Rede



- Repetidores (hubs)
- Comutadores (switches)
- Encaminhadores (routers)

# Fundamentos de Encaminhamento



- Necessidade de políticas de encaminhamento
- Encaminhamento
  - Estático
  - Dinâmico
- Encaminhamento
  - Centralizado
  - Distribuído

# Encaminhamento estático



- As tabelas são definidas com base no conhecimento ‘a priori’ da rede
- As tabelas raramente são alteradas
- Tipicamente, as tabelas são introduzidas manualmente nos routers pelo administrador de sistemas

**Utilização**

- Redes pequenas
- Redes com poucas alterações de topologia
- Redes com uma única ligação ao exterior
- Redes sem caminhos redundantes

# Encaminhamento Dinâmico



- As tabelas de encaminhamento são alteradas de forma automática (daemons)
- A informação recebida pelos daemons é trocada utilizando protocolos de encaminhamento

**Utilização**

- Redes de grande dimensão e de topologias complexas
- Redes com frequentes alterações de topologia
- Redes com caminhos redundantes, com diferentes características e custos, que se alteram ao longo do tempo

# Vantagens do encaminhamento dinâmico



- Adaptação a alterações da topologia da rede
- Reacção a condições de congestão da rede
- Aumento do desempenho da rede
- Sobrevivência a falhas
- Contribuição para a qualidade de serviço

# Desvantagens do Encaminhamento dinâmico



- Overhead (protocolos para disseminação da informação de routing)
- Maior complexidade (decisões de routing, protocolos, processamento nos routers)
- Consistência da informação de routing
  - muitas mensagens: boa actualização, muito overhead
  - poucas mensagens: má actualização, pouco overhead
- Tempos de reacção muito curtos podem provocar oscilações
- Mais cuidado para evitar ciclos (loops)



# Deepweb

# Definitions

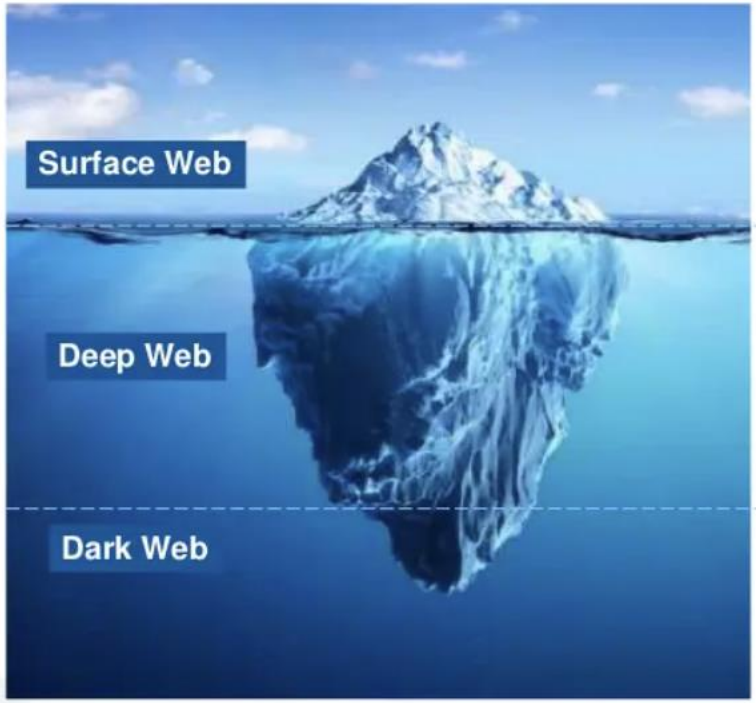
- World Wide Web / Surface Web /Clear Net
- Deep Web
- Dark Web



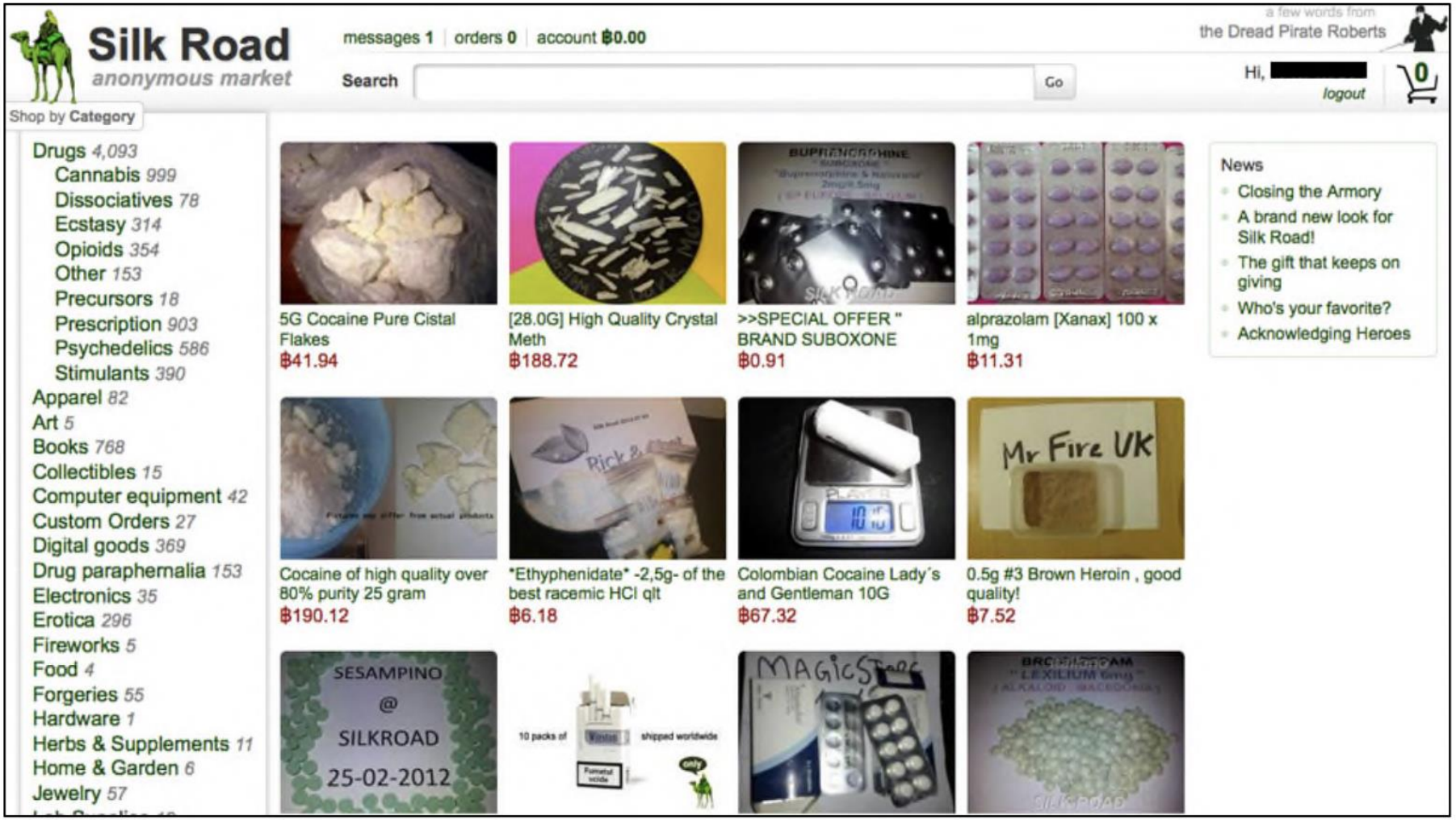
- The same infrastructure: TCP/IP
- Application Layer


Surface Web
Exploring The Hidden Internet
4%
Deep Web
90%
Legal Documents
Government Records
Scientific Reports
Academic Records
Financial Records
Dark Web
6%
Drugs
Tor
Illegal Info

# “The iceberg”




Google
amazon
ebay
YouTube
WIKIPEDIA
The Free Encyclopedia
PayPal
Bank of America
HSBC
Dropbox
Silk Road
anonymous marketplace
WikiLeaks
Wall Market




*http://www.thehiddenwiki.net/purchasing-on-silk-road-a-trip-report/

# Definitions

## TOR

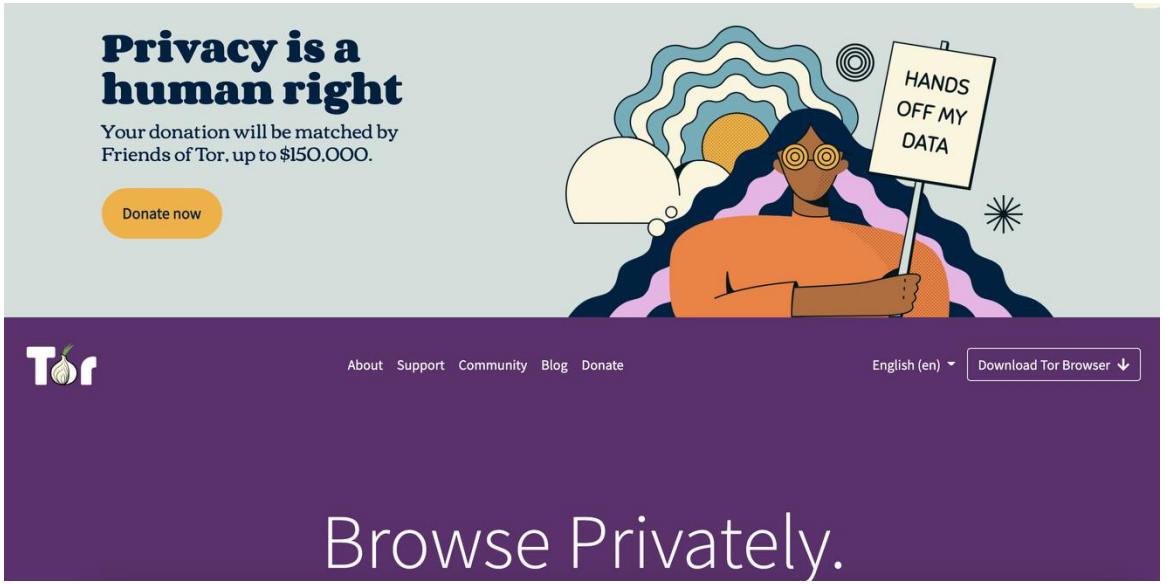
- Short for “The Onion Router” (TOR);
- The most famous anonymous network, which can be accessed using the TOR browser;
- TOR browser allows users to surf the dark web anonymously by directing traffic through a network of intermediaries (adding layers).

# Timeline of the Deep Web

- 2002 – Tor Network developed by US Naval Research Laboratory
- 2003 – The I2P peer-to-peer network developed
- 2006 – Tor becomes a non-profit organization
- 2009 – “Satoshi Nakamoto” started Bitcoin digital currency
- 2011 – Silk Road Market launches
- 2012 – Experts estimate earnings of $22M for SR
- 2013 – Silk Road shut down (earnings > $1B)
- 2014 – Other dark web markets proliferate (Agora, Evolution)
- 2015 – Ross Ulbricht sentenced to life; Evolution exit scam

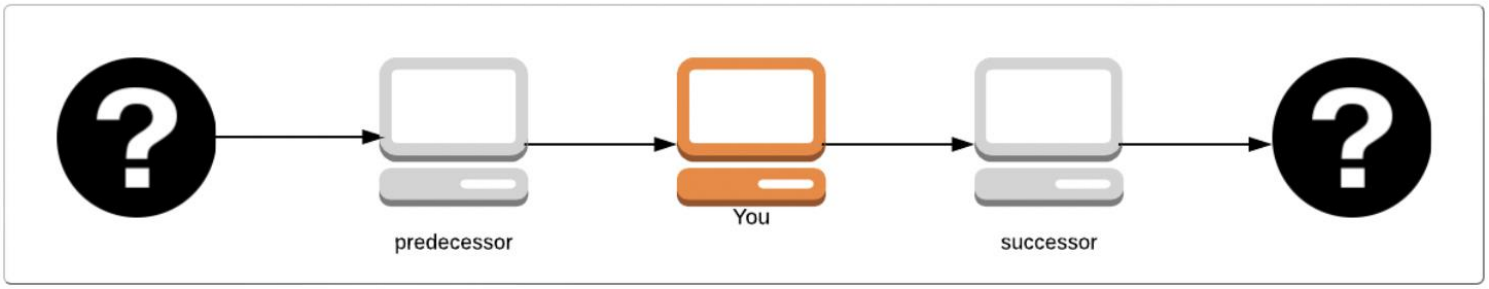
# Tor

- https://www.torproject.org/
- Tor Browser
- Tor Net (Onion)



# Tor

- Onion Routing is a distributed overlay network designed to anonymise TCP-based applications
- Clients choose a *path *through the network and build a *circuit* where each onion router in the path knows the predecessor and the successor, but no other nodes in the circuit.
- No one knows what data is being sent until it reaches the last node in the path; who knows the data but doesn't know who sent it. The second to last node in the path doesn't know what the data is, only the last node in the path does.



# Tor Relays

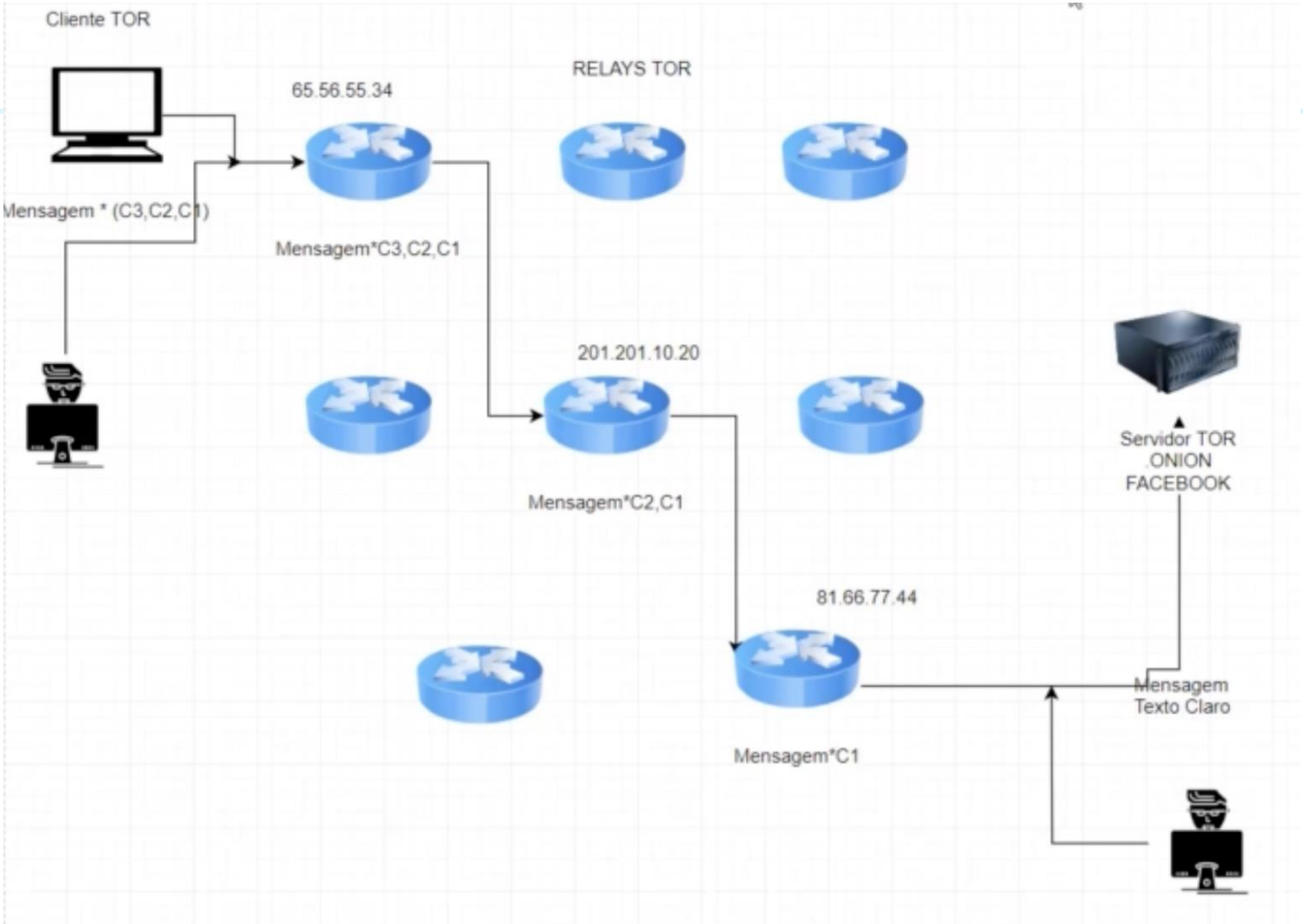
- They are Routers /Crossing points (ponto de passagem)
- Type
  - Middle Relays (public)
  - Exit Relay (final)
  - Bridges (hidden)

# Installing

- Tor Browser
  - Firefox
  - Java

# How to attack Tor?

- Large organisations with expansive resources run servers to attempt to be the first and last nodes in the network.
- If the organisation's server is the first node, it knows who sent the message.
- If the organisation server is the last node, it knows the final destination and what the message says.

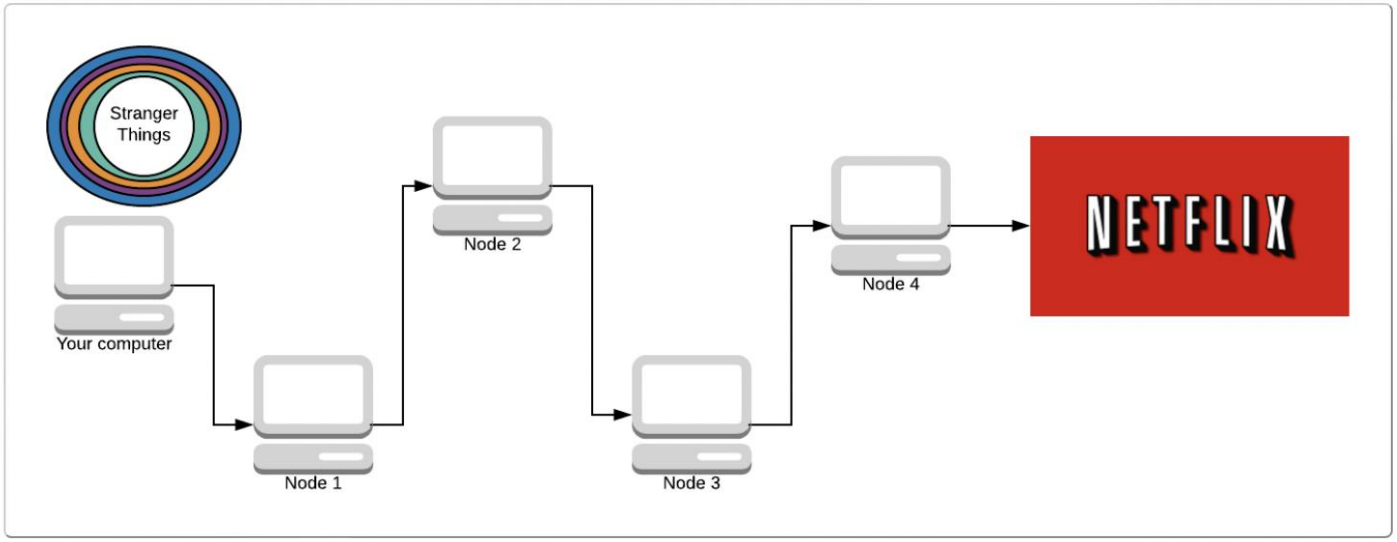

Cliente TOR
RELAYS TOR
65.56.55.34
Mensagem * (C3,C2,C1)
Mensagem*C3,C2,C1
201.201.10.20
Mensagem*C2,C1
Servidor TOR
ONION
FACEBOOK
81.66.77.44
Mensagem
Texto Claro
Mensagem*C1

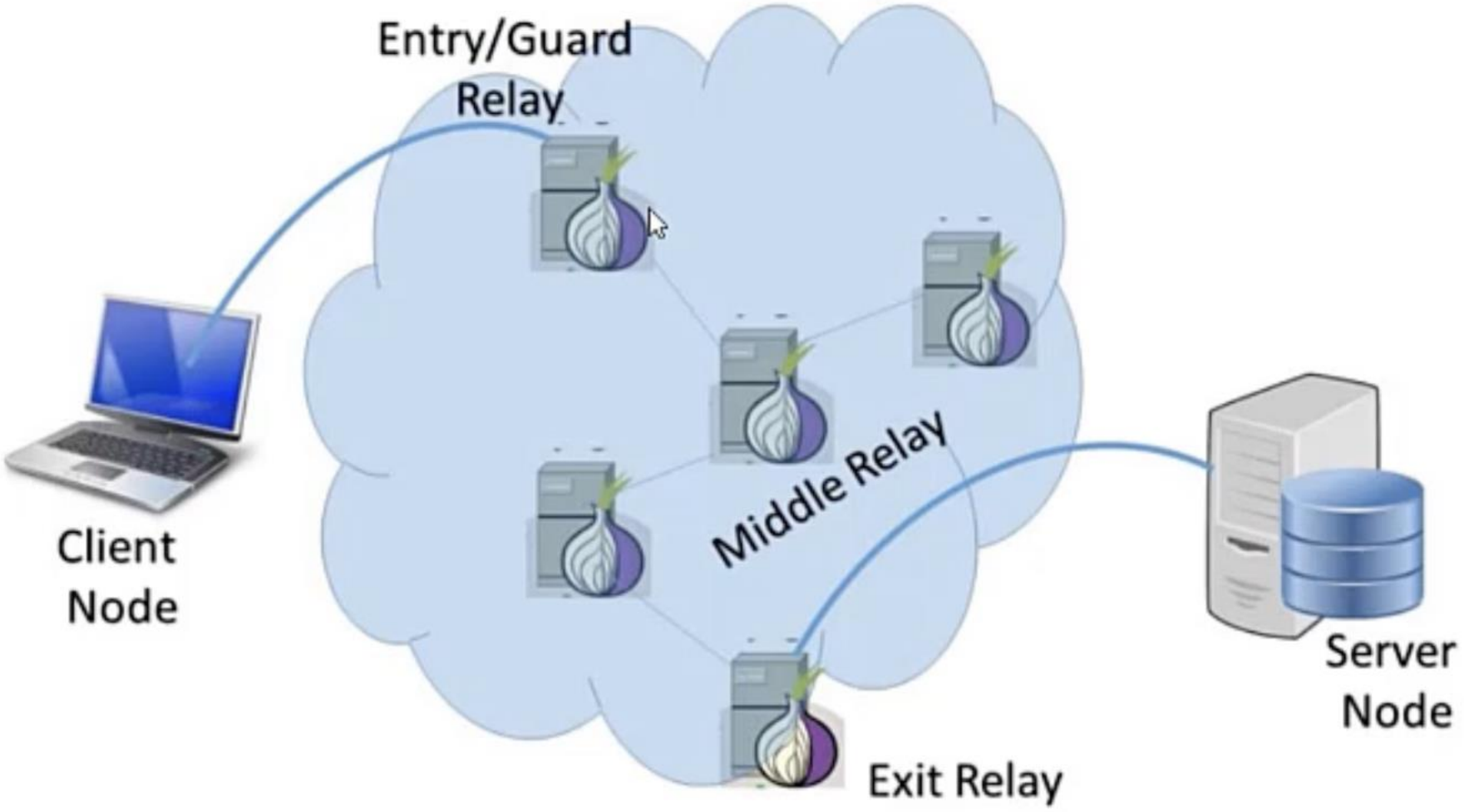
# Tor

- Each packet flows down the network in *fixed-size cells*. These cells have to be the same size so none of the data going through the Tor network looks suspiciously big.
- These cells are unwrapped by a **symmetric key** at each router and then the cell is relayed further down the path.
- Tor needs a lot of users to create anonymity, if Tor was hard to use new users wouldn't adopt it so quickly. Because new users won't adopt it, Tor becomes less anonymous. By this reasoning it is easy to see that usability isn't just a design choice of Tor but a *security requirement* to make Tor more secure.

# Tor

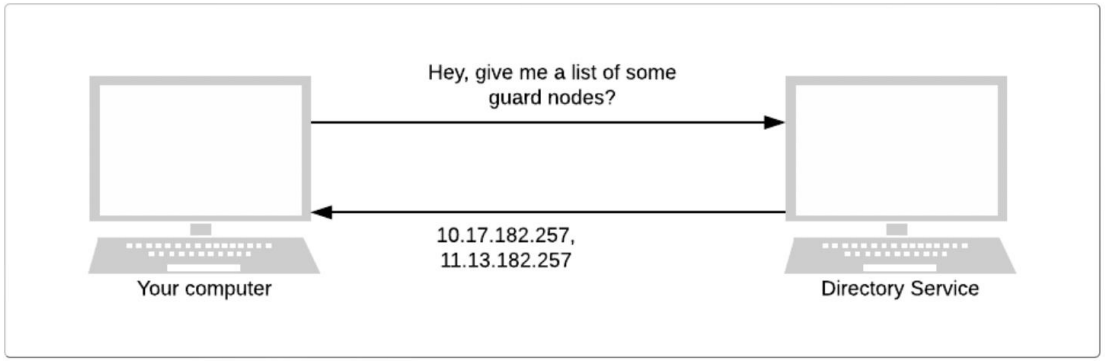
- **Symmetric keys between neighbor nodes**
- The encryption used is normally AES with the key being shared via Diffie-Hellman.

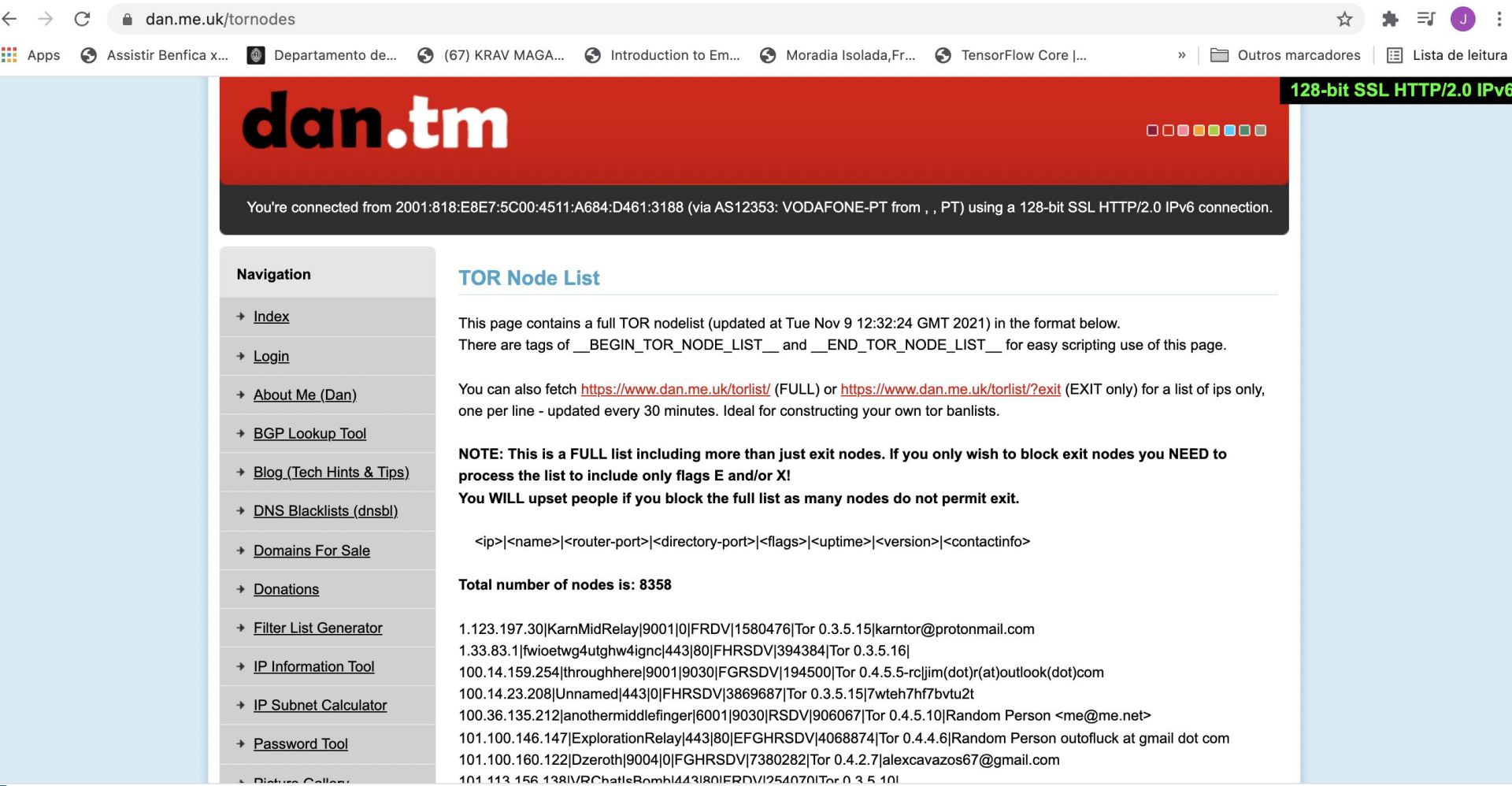



Entry/Guard
Relay
Middle Relay
Client
Node
Server
Node
Exit Relay

# Directory Node

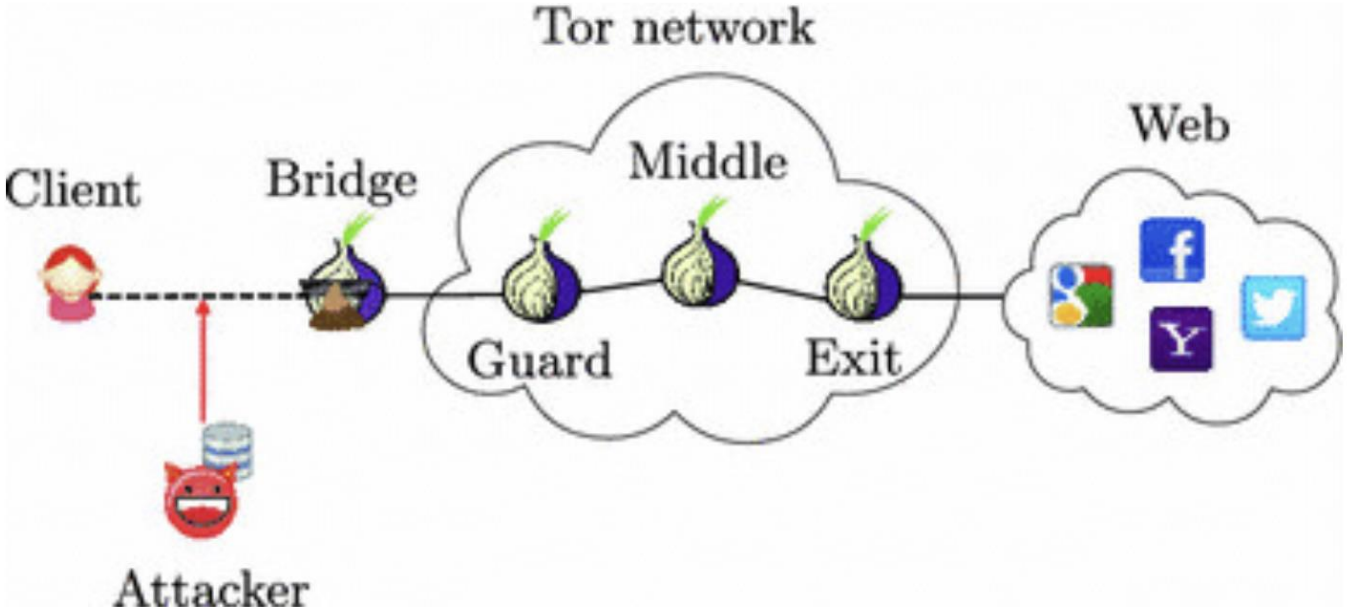
- The state of the Tor network is tracked and publicised by a group of 9 trusted servers (as of 2019) known as *directory nodes*. Each of which is controlled by a different organisation.
- Directory nodes maintain a list of currently running relays (publicly listed node in the Tor network). Once per hour directory nodes publish a consensus together. The consensus is a single document compiled and voted on by each directory node.
- When a Tor user (a client or a node) wants to know the current state of the network, it asks a directory node. -> Potential Problem
- Instead of asking a single directory node for its opinion, clients asks every directory node and combined their opinions into a consensus.



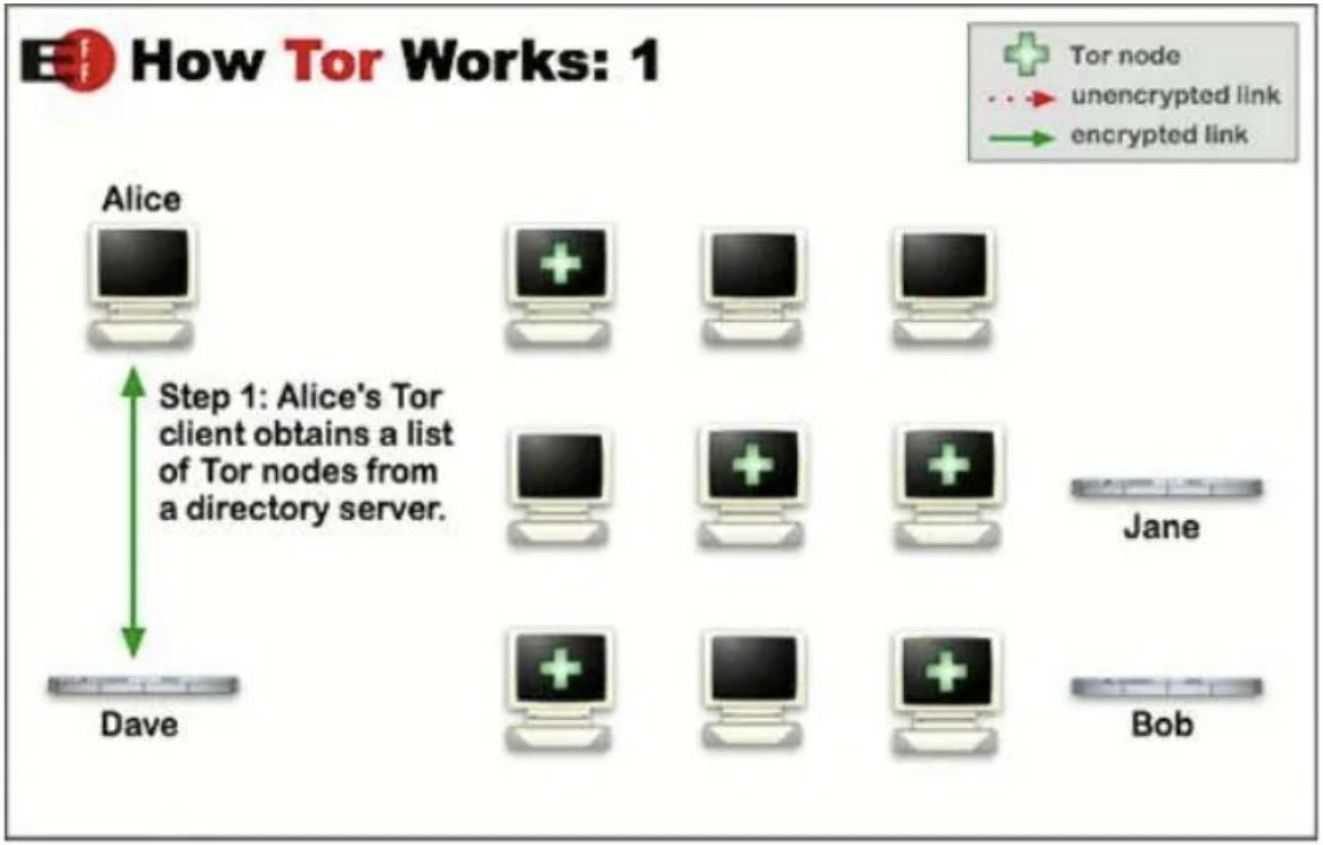

dan.me.uk/tornodes
Apps
Assistir Benfica x...
Departamento de...
(67) KRAV MAGA...
Introduction to Em...
Moradia Isolada,Fr...
TensorFlow Core |...
Outros marcadores
Lista de leitura
dan.tm
128-bit SSL HTTP/2.0 IPv6
You're connected from 2001:818:E8E7:5C00:4511:A684:D461:3188 (via AS12353: VODAFONE-PT from , , PT) using a 128-bit SSL HTTP/2.0 IPv6 connection.
Navigation
Index
Login
About Me (Dan)
BGP Lookup Tool
Blog (Tech Hints & Tips)
DNS Blacklists (dnsbl)
Domains For Sale
Donations
Filter List Generator
IP Information Tool
IP Subnet Calculator
Password Tool
TOR Node List
This page contains a full TOR nodelist (updated at Tue Nov 9 12:32:24 GMT 2021) in the format below.
There are tags of __BEGIN_TOR_NODE_LIST__ and __END_TOR_NODE_LIST__ for easy scripting use of this page.
You can also fetch https://www.dan.me.uk/torlist/ (FULL) or https://www.dan.me.uk/torlist/?exit (EXIT only) for a list of ips only, one per line - updated every 30 minutes. Ideal for constructing your own tor banlists.
NOTE: This is a FULL list including more than just exit nodes. If you only wish to block exit nodes you NEED to process the list to include only flags E and/or X!
You WILL upset people if you block the full list as many nodes do not permit exit.
<ip>|<name>|<router-port>|<directory-port>|<flags>|<uptime>|<version>|<contactinfo>
Total number of nodes is: 8358
1.123.197.30|KarnMidRelay|9001|0|FRDV|1580476|Tor 0.3.5.15|karntor@protonmail.com
1.33.83.1|fwioetwg4utghw4ignc|443|80|FHRSDV|394384|Tor 0.3.5.16|
100.14.159.254|throughhere|9001|9030|FGRSDV|194500|Tor 0.4.5.5-rc|jim(dot)r(at)outlook(dot)com
100.14.23.208|Unnamed|443|0|FHRSDV|3869687|Tor 0.3.5.15|7wteh7hf7bvtu2t
100.36.135.212|anothermiddlefinger|6001|9030|RSDV|906067|Tor 0.4.5.10|Random Person <me@me.net>
101.100.146.147|ExplorationRelay|443|80|EFGHRSDV|4068874|Tor 0.4.4.6|Random Person outofluck at gmail dot com
101.100.160.122|Dzeroth|9004|0|FGHRSDV|7380282|Tor 0.4.2.7|alexcavazos67@gmail.com

# Tor Bridge

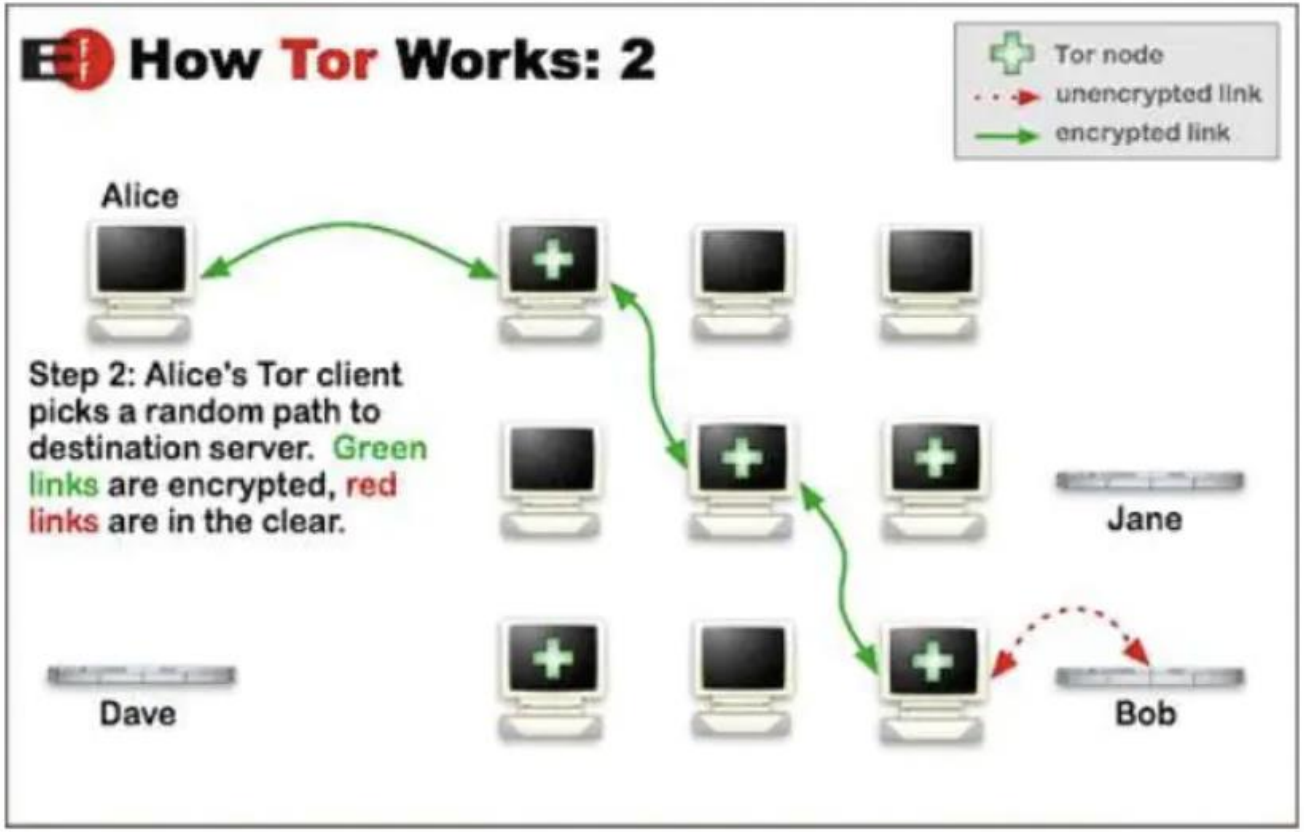
- IP address is not public
- One way to get bridges is to send an email to bridges@torproject.org.



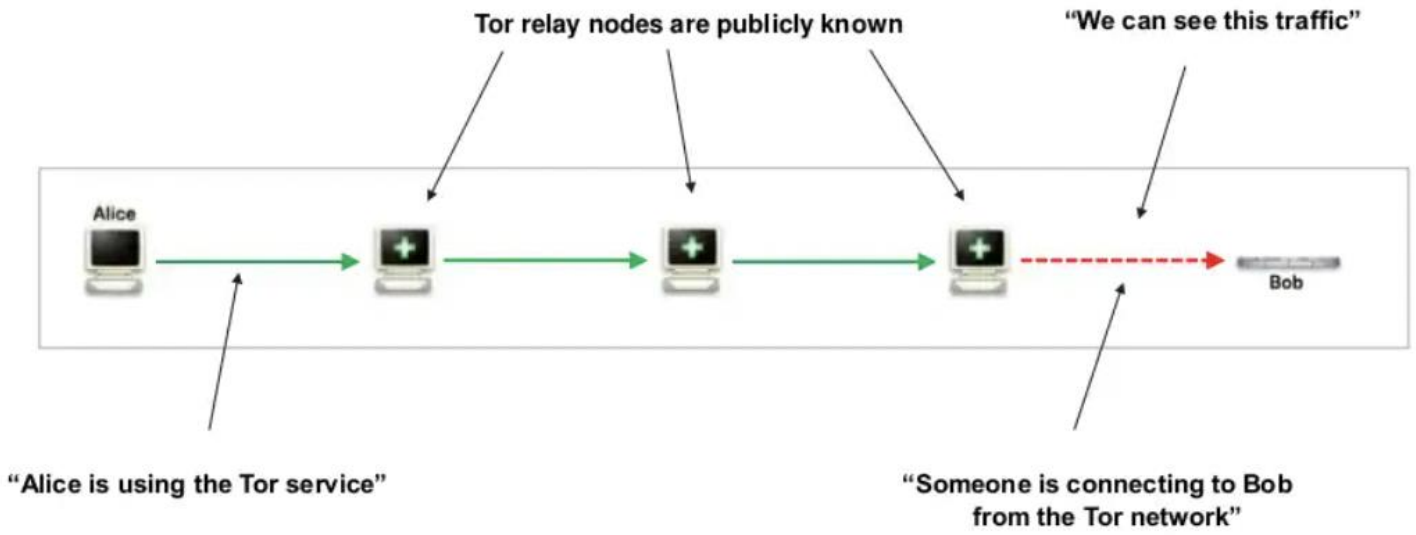
# TOR


How Tor Works: 1
Tor node
unencrypted link
encrypted link
Alice
Step 1: Alice's Tor client obtains a list of Tor nodes from a directory server.
Jane
Dave
Bob

# TOR


How Tor Works: 2
Tor node
unencrypted link
encrypted link
Alice
Step 2: Alice's Tor client picks a random path to destination server. Green links are encrypted, red links are in the clear.
Jane
Dave
Bob

# How anonymous?



# Who owns the Tor nodes?

Volunteers.



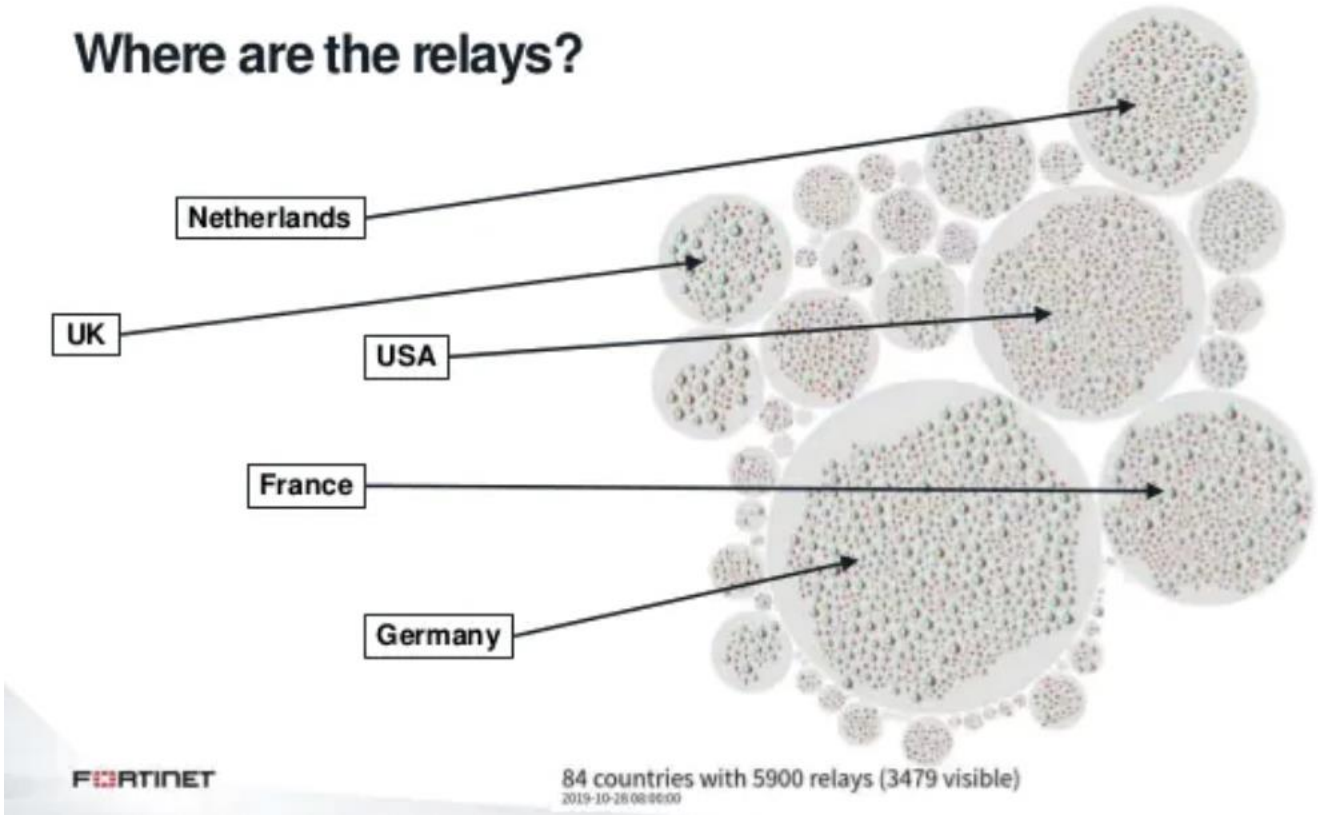
Often universities and other institutions

Most people host Relay or Guard (Entry) nodes

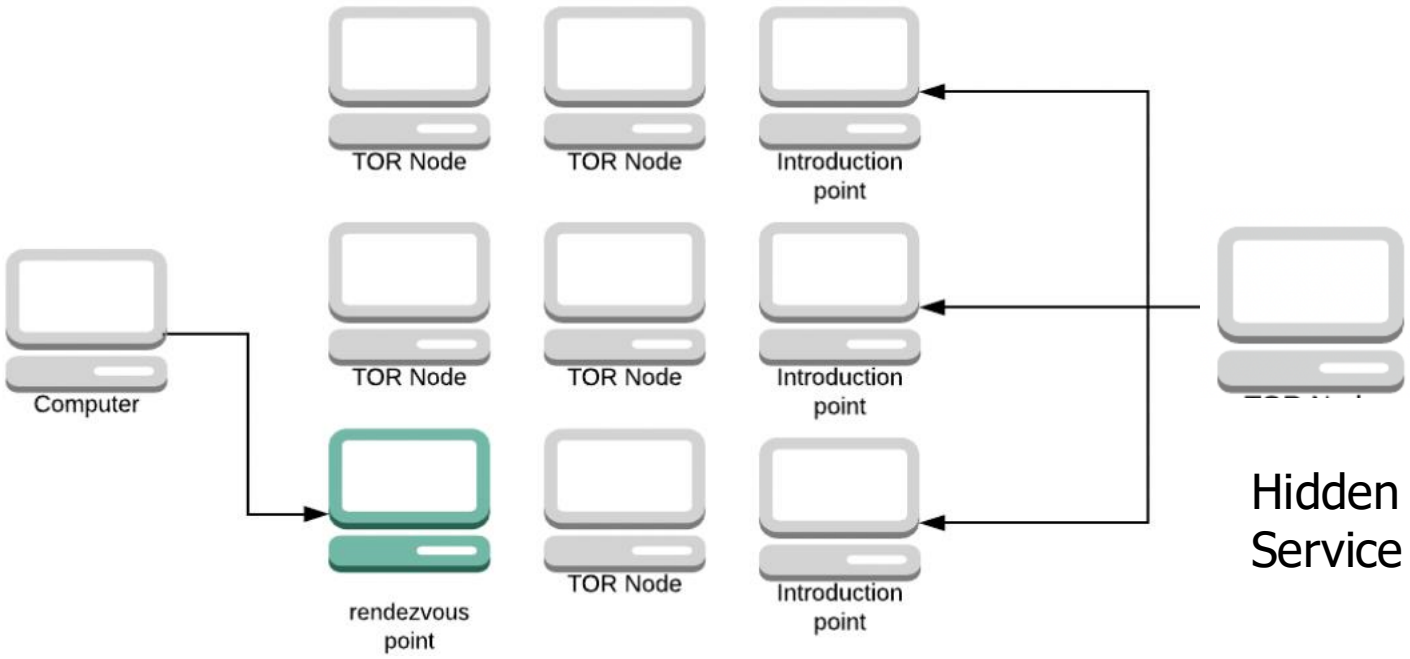
Nodes cannot become guards unless they are stable, and have at least 2Mbytes/s bandwidth

Running an Exit node opens up the potential of receiving abuse complaints

Exit nodes are often blocked by providers or website owners


Where are the relays?
Netherlands
UK
USA
France
Germany
FORTINET
84 countries with 5900 relays (3479 visible)
2019-10-28 08:00:00

# Tor Hidden Services





Questions?